Those two sub-headers instruct PHP to take all the bytes it finds up to the next occurrence of the boundary string (multipart request bodies are always terminated by a final instance of the boundary string) and treat them as a single file. The resulting bytes are then written to the `tmp_name` filepath as detailed in the `$_FILES` data.

Now, before you go digging around the `/tmp` directory of your cli-server container for the uploaded file, bear in mind that it's a temporary file and PHP will remove it once the request is processed and the script terminates. Occasionally these files aren't removed if a script terminates unexpectedly, but in our case we're just looking at a superglobal data structure; there really isn't much to go wrong. We'll need to do a little more work to persist the upload to the local filesystem. There are two functions that interact with file uploads: `is_uploaded_file()` is a simple function that tests if a file was uploaded via POST; `move_uploaded_file()` will move an uploaded file (and will also perform the same verification). It is recommended to use these functions to interact with file uploads – rather than alternative PHP filesystem functions – because they will ensure the target file is a genuine POST upload. As before, create a new index.php file with the following code…

```
// index.php

$tmpFile = $_FILES['file']['tmp_name'];
var_dump(is_uploaded_file($tmpFile));
var_dump(is_uploaded_file('/etc/passwd'));
var_dump(move_uploaded_file($tmpFile, __DIR__ . '/uploaded.jpg'));
var_dump(is_uploaded_file($tmpFile));
```

… and run the following curl command, which assumes that you are working in the same directory from which you launched your cli-server container, and there is a `./random.jpg` image file available:

```
$ curl -F 'file=@./random.jpg' http://localhost:8080/index.php
bool(true)
bool(false)
bool(true)
bool(false)
```

The code in `index.php` calls `is_uploaded_file()` first to verify that `$_FILES['file']['tmp_name']` is a genuine POST upload (which returns `true`, so it is), then a quick demonstration that a critical system file `/etc/passwd` is not (this time the